

Vulnerability 2: Union-Based SQL Injection in Product Search

CWE: CWE-89 - Improper Neutralization of Special Elements used in an SQL Command

OWASP: A03:2021 - Injection

CVSS: 8.6 (High)

Location: routes/search.ts

Component: Backend API - Product search endpoint (GET /rest/products/search)

Description

The product search endpoint builds its SQL query by directly interpolating the user-supplied search term (req.query.q) into the query string using JavaScript template literals. Because there is no parameterization, an attacker can inject arbitrary SQL syntax through the q parameter. Unlike the login SQLi which enables authentication bypass, this vulnerability enables data exfiltration. An attacker can use UNION SELECT to retrieve data from any table in the database, including user email addresses and password hashes.

The LIKE '%...%' structure of the original query means the attacker must first close the surrounding quotes and parentheses before appending their payload.

Vulnerable code (Backend):

`models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)`

```
1 export function searchProducts () {
2   return (req: Request, res: Response, next: NextFunction) => {
3     let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
4     criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
5     models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%' ) AND deletedAt IS NULL) ORDER BY name`)
6     .then(([products]: any) => {
7       const dataString = JSON.stringify(products)
8       if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-snippet hide-start
9         let solved = true
10        UserModel.findAll().then(data => {
11          const users = utils.queryResultToJson(data)
12          if (users.data?.length) {
13            for (let i = 0; i < users.data.length; i++) {
14              solved = solved && utils.containsOnEscaped(dataString, users.data[i].email) && dataString.includes(users.data[i].password)
15              if (!solved) {
16                break
17              }
18            }
19            if (solved) {
20              challengeUtils.solve(challenges.unionSqlInjectionChallenge)
21            }
22          }
23        }).catch((error: Error) => {
24          next(error)
25        })
26      }
27    })
28  }
29 }
```

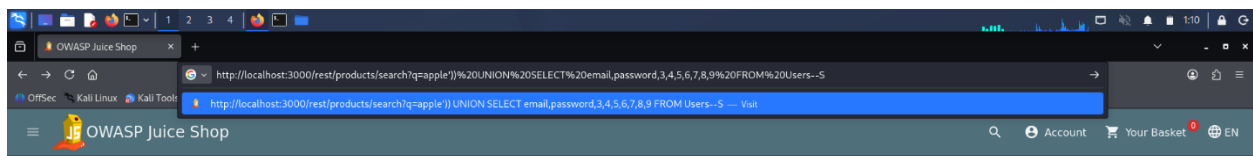
```
1 models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%' ) AND deletedAt IS NULL) ORDER BY name`)
```

The `${criteria}` value comes from `req.query.q` and is concatenated directly into the SQL string. The database cannot distinguish between developer SQL syntax and attacker-supplied input.

Exploit payload: ')) UNION SELECT email,password,3,4,5,6,7,8,9 FROM Users—

[http://localhost:3000/rest/products/search?q=apple'\)\)%20UNION%20SELECT%20email,password,3,4,5,6,7,8,9%20FROM%20Users--](http://localhost:3000/rest/products/search?q=apple'))%20UNION%20SELECT%20email,password,3,4,5,6,7,8,9%20FROM%20Users--)

1. Navigate to <http://localhost:3000/#/search> or use the search bar.
2. Submit the payload as the search term.
3. The response renders the injected user data inside the product cards.



Resulting query executed by the database:

SELECT * FROM Products WHERE ((name LIKE '%'))

UNION SELECT email,password,3,4,5,6,7,8,9 FROM Users--%' OR description LIKE '%...%')
AND deletedAt IS NULL) ORDER BY name

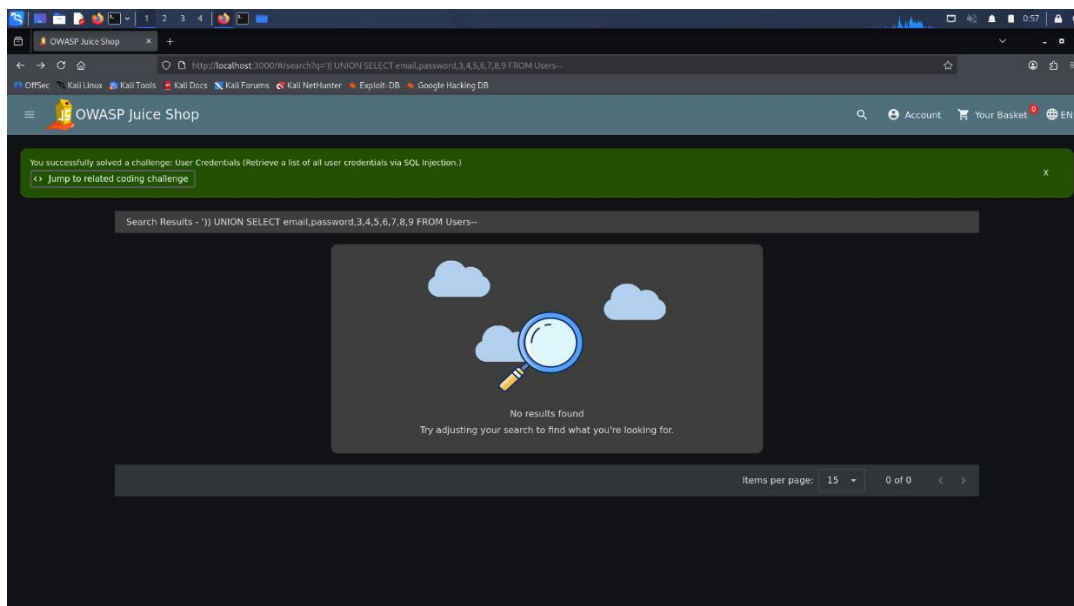
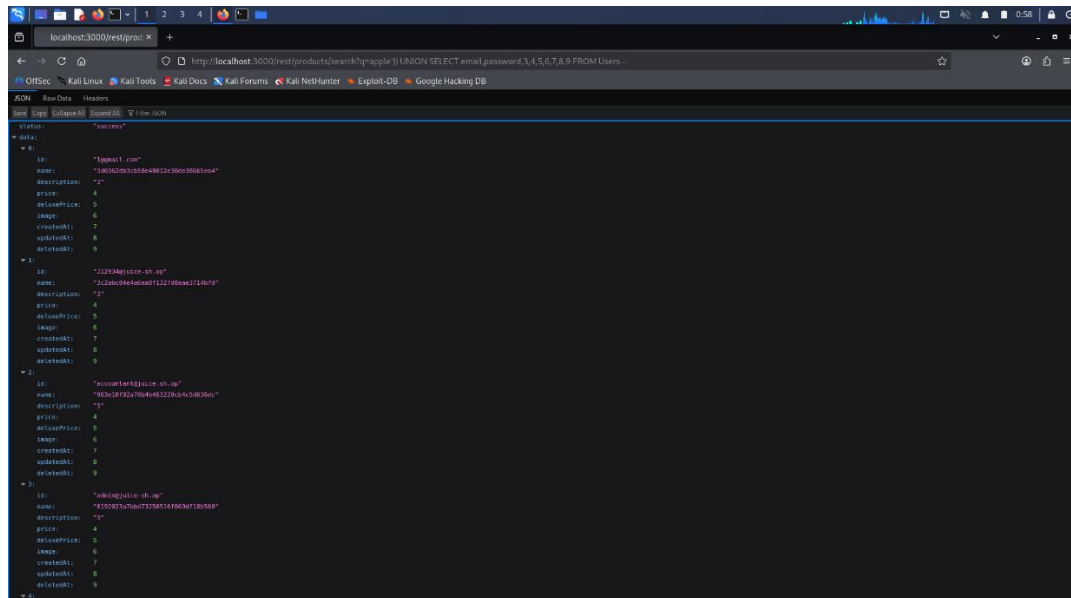
The Vulnerability:

Payload part	Effect on the query
'))	Closes the LIKE clause's string and the outer parenthesis group
UNION SELECT email,password,...	Appends a new query returning user credentials
,3,4,5,6,7,8,9	Padding values to match the 9-column structure of the Products table
FROM Users	Targets the Users table
--	Comments out the remainder of the original query (AND deletedAt IS NULL) ORDER BY name)

The first part returns no products, the UNION SELECT returns user emails and password hashes, and the rest of the original query is commented out.

Evidence - Before Fix

- Navigate to the product search page (<http://localhost:3000/#/search>) or send the request directly to the API endpoint GET `/rest/products/search?q=<payload>`.
- The response returns product search results containing user email addresses and password hashes injected via the UNION SELECT clause data that should never appear in the product listing.



SAST Detection - Before Fix

Tool: Semgrep

Rule: sequelize-template-literal-injection

Run against the unmodified routes/search.ts, Semgrep flags the interpolated query. Capture the finding count and the specific rule ID and line number flagged.

```
(semgrep-venv)-(kali㉿kali)-[~/juice-shop-lab/juice-shop]
$ cd ~/juice-shop-lab/juice-shop
semgrep scan \
  --config=.semgrep-rules/ \
  --metrics=off \
  routes/search.ts \
  --json --output=semgrep-search-before.json
info: Required file matching /"hacking-instructor-.+\.js$/ is available (0.00s)
warn: https://www.alchemy.com/ is not reachable (warn)
warn: "Honey Pot" challenge will not work as intended
warn: "Waller Depletion" challenge will not work as intended
? Loading rules ... variable ALCHEMY_API_KEY is not present (warn)
Scanning 1 file (only git-tracked) with 1 Code rule:
warn: "Waller Depletion" challenge will not work as intended
? CODE RULES https://howto-web3.owasp-juice.shop for instructions
w Scanning 1 file. //localhost:11434/v1 is not reachable (warn)
warn: "Chain Prompt Injection" challenge will not work as intended
w SUPPLY CHAIN RULES Manipulation" challenge will not work as intended
warn: "AI Debugging" challenge will not work as intended with
w No rules to run: Extraction" challenge will not work as intended
info: Check https://howto-llm.owasp-juice.shop for instructions
info: Check https://howto-llm.owasp-juice.shop for instructions
info: Check https://howto-llm.owasp-juice.shop for instructions
? PROGRESS 4-star challenge solved in 0.00s (User Credentials)
info: Cheat score for 4-star challenge solved in 0.00s
100% 0:00:00
?
ssh: suspended - PORT=3000 npm start

~/juice-shop-lab/juice-shop

Scan Summary
✓ Scan completed successfully.
• Findings: 1 (1 blocking)
• Rules run: 1
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 1 rule on 1 file: 1 finding.
```

Secure Coding Fix

Replace the string interpolation with a **parameterized query**. In Sequelize, this is achieved using (?) placeholders and the replacements array.

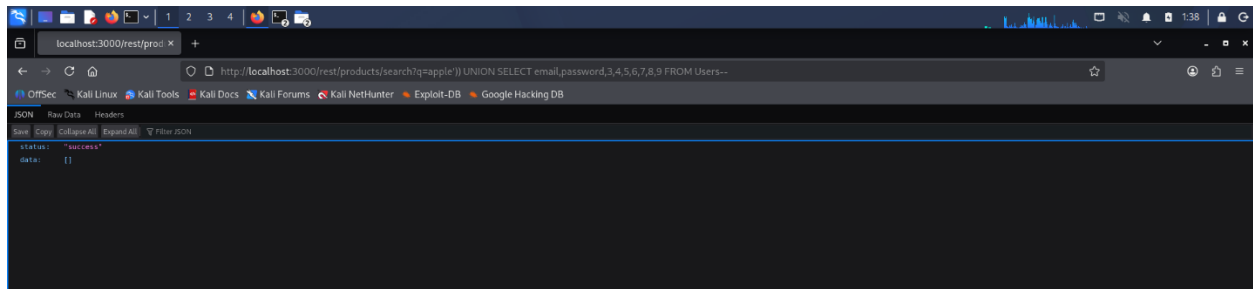
```
models.sequelize.query('SELECT * FROM Products WHERE ((name LIKE ? OR description LIKE ?) AND deletedAt IS NULL) ORDER BY name', {replacements: ['%${criteria}%'], type: QueryTypes.SELECT})
```

```
1 export function searchProducts () {
2   return (req: Request, res: Response, next: NextFunction) => {
3     let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
4     criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
5     models.sequelize.query(
6       'SELECT * FROM Products WHERE ((name LIKE ? OR description LIKE ?) AND deletedAt IS NULL) ORDER BY name',
7       {
8         replacements: ['%${criteria}%', '%${criteria}%'],
9         type: QueryTypes.SELECT
10      }
11    )
12    .then((products: any) => {
13      const dataString = JSON.stringify(products)
14      if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-snippet hide-start
15        let solved = true
16        UserModel.findAll().then(data => {
17          const users = utils.queryResultToJson(data)
18          if (users.data?.length) {
19            for (let i = 0; i < users.data.length; i++) {
20              solved = solved && utils.containsOrEscaped(dataString, users.data[i].email) && dataString.includes(users.data[i].password)
21              if (!solved) {
22                break
23              }
24            }
25            if (solved) {
26              challengeUtils.solve(challenges.unionSqlInjectionChallenge)
27            }
28          }
29        }).catch((error: Error) => {
30          next(error)
31        })
32      }
33    })
34  }
```

```
1 models.sequelize.query('SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name')
```

Why this works: With parameterization, the payload ')) UNION SELECT ...--' is sent to the database as a literal string value for the LIKE comparison. The database searches for product names or descriptions containing that exact string, which won't match. The UNION SELECT never executes, and no user data is returned.

Evidence - After Fix



SAST Detection - After Fix

Re-running the same scan on the patched search.ts shows the SQL injection finding is gone. Capture the after-fix finding count.

```
(semgrep-venv)-(kali@kali)-[~/juice-shop-lab/juice-shop]
$ cd ~/juice-shop-lab/juice-shop
semgrep scan \
  --config .semgrep-rules/ \
  --metrics=off \
  routes/search.ts \
  --json --output=semgrep-search-after.json

ooo
Semgrep CLI

Scanning 1 file (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 1 file.

SUPPLY CHAIN RULES
No rules to run.

PROGRESS
100% 0:00:00

Scan Summary
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 1
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 1 rule on 1 file: 0 findings.

✦ If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/

(kali@kali)-[~/juice-shop-lab/juice-shop]
$
```

Before / After Summary

Metric	Before Fix	After Fix
Semgrep finding count on search.ts	1	0
Exploit ')) UNION SELECT ...--	User credentials returned	Login rejected
SQL query construction	String interpolation	Parameterized (? + replacements)
Root cause present	Yes	No
CWE-89 compliant	No	Yes